MultiQT notes

November 10, 2026

Notation

Unless stated otherwise, fix one disease, location and forecast horizon; their indices are suppressed.

Forecasts and indices. The indices s, t order forecast reference weeks (`reference_date`, Saturday); j also denotes a historical step inside a re-run. The outcome y_t belongs to the target week of forecast q_t, identified by `target_end_date`, rather than necessarily to its reference week. The quantile levels are $\mathcal{A} = \{\alpha_1 < \cdots < \alpha_m\} \subset (0, 1)$, with $m = |\mathcal{A}|$. Write $b_t = (b_t^\alpha)_{\alpha\in\mathcal{A}}$ for the base forecasts and $q_t = (q_t^\alpha)_{\alpha\in\mathcal{A}}$ for the recalibrated forecasts. Unless a transformation is shown explicitly, forecasts and outcomes are in outcome units (hospital counts in the code). The methodological recursions first issue q_t, then observe y_t and update for the next step. The coding section specifies when delayed or revised reports become available instead.

Loss and projections. The indicator is $\mathbf{1}\{\cdot\}$ and $x_+ = \max(x, 0)$. Quantile loss takes the prediction as its first argument and the observation as its second:

$$\rho_\alpha(z, r) = \alpha(r - z)_+ + (1 - \alpha)(z - r)_+. \tag{1}$$

The ordered-quantile cone and Euclidean projection are

$$\mathcal{K} = \{q \in \mathbb{R}^m : q^{\alpha_1} \le \cdots \le q^{\alpha_m}\}, \qquad \Pi_C(x) = \arg\min_{z\in C} \|z - x\|^2,$$

where C is nonempty, closed and convex and $\|\cdot\|$ is the Euclidean norm. Assume $b_t \in \mathcal{K}$. For a positive-definite matrix W, write

$$\Pi_{\mathcal{K}}^W(v) = \arg\min_{q\in\mathcal{K}} (q - v)^\top W(q - v).$$

Q_p denotes an empirical p-quantile; I denotes the identity matrix.

Scores and tracker states. $S_t(y)$ is a score function, $r_t = S_t(y_t)$ its observed value, and c_t its issued threshold in score units. The target coverage is α, with target miscoverage $\delta = 1 - \alpha$; empirical coverage at a step is $\mathrm{cov}_t^\alpha = \mathbf{1}\{y_t \le q_t^\alpha\}$. The superscripts std and stu distinguish standard and studentized methods. A tilde marks hidden, accumulated state; the untilded state is played after projection. Thus $\tilde\theta_t, \theta_t$ are offsets in outcome units, while $\tilde u_t, u_t$ are thresholds in studentized units. The finite, positive scales are σ_t^α, collected in

$$A_t = \mathrm{diag}(\sigma_t^{\alpha_1}, \ldots, \sigma_t^{\alpha_m});$$

σ_t denotes a scale common to all levels and σ a constant scale. The tracker step size is η (or η_t when varying), and κ is a rate multiplier. The selected loss subgradient is written g_α; its form is derived in (**??**).

Reports, feedback and re-runs. The date ι_t is the information cutoff for forecast q_t: normally the Wednesday of its reference week, with actual generation dates used where known. The notation $y_s^{[v]}$ means the reported count for the target week of q_s in the latest report dated on or before date v (the archive's `version` field). A valid report is finite and nonnegative. Its first valid publication date is ν_s, and $d \geq 0$ is an additional delay in calendar weeks. The symbols $m_s^{(d)}$, $\tau_d(s)$ and $\mathcal{R}_t^{(d)}$ denote the maturity date, the index of the round that consumes feedback for q_s, and the set of rows consumed before issuing q_t, respectively; their scheduling rules are given in (**??**)–(**??**). The set $I_{20}^{(d)}$ contains the first 20 consumed forecast rows in release order, with their stateful report values

$$\bar{y}_s^{(d)} = y_s^{[\iota_{\tau_d(s)}]}, \qquad s \in I_{20}^{(d)}. \tag{2}$$

For forecasts and states, a superscript $[t]$ instead identifies the re-run at reference-week index t: $q_s^{[t]}$ is the forecast for row s reconstructed inside that re-run. Thus v in $y_s^{[v]}$ is a report-cutoff date, while t in $q_s^{[t]}$ is a re-run index. Feedback respects the cutoff convention, subject to the retrospective initialization described in Section **??**.

Implementation scales. The tracker transformation is $\psi(x) = x$ or $\psi(x) = \log(1 + x)$. The count divisor $c_{\text{scale}} > 0$ is population divided by 100,000 by default, so y/c_{scale} is a rate per 100,000 residents. The fixed reference constants S_0 (initial error scale) and L_0 (expert loss normalization) are defined in (**??**) and (**??**); S_0 is separate from the time-indexed score function S_t. Their numerical floors are ε_S and ε_L.

Experts and weighting. Expert k issues $f_{k,t}^{\alpha}$ at level α in forecast round t. The index n counts processed expert-weight updates, not calendar weeks; $s(n)$ is the forecast row whose feedback is consumed, and Y_n is the report used in that update. For a fixed bank of K experts, $w_{k,n}^{\alpha}$ is the weight immediately before update n and λ_n^{α} its learning rate; $\bar{w}_{k,n+1}^{\alpha}$ is the weight after the exponential step but before optional sharing. Common weights and rates omit α: $w_{k,n}$ and λ_n. Suppress time indices when only one forecast or update is considered; $\hat{q}$ is the mixture before any crossing repair. The unnormalized expert losses are $L_{k,n}^{\alpha}$; $\ell_{k,n}^{\alpha}$ are their normalized versions, and $L_{k,n}$ and $\ell_{k,n}$ are the corresponding means over levels. The second moments are M_n^{α} or M_n, the lookback is H, the starting rate is λ_{start}, and γ is the optional fraction of weight shared uniformly among experts. Superscripts raw and norm distinguish calculations without and with L_0 normalization. For the soft switch, $G_t \in [0.05, 0.95]$ is the declining-context gate at issuance; $w_{k,t}^{B}$ and $w_{k,t}^{D}$ are the current weights in its baseline and declining banks, and $w_{k,t}^{\text{mix}}$ are the combined weights used for that forecast.

1 Methodological notes

1.1 Studentized score

Writing the score as a signed residual gives the usual additive MultiQT correction. Studentizing that residual gives multiplicatively scaled offsets, or, equivalently, offsets tracked in scaled coordinates. A changing scale rescales the accumulated correction as well as each new increment. If the scale decreases after a peak, it can shrink the outcome-scale contribution of a positive correction accumulated during an upswing. This is the motivation; the hidden accumulated state is not erased.

With quantile-specific scales, the ordering constraint defines a diagonally transformed, shifted isotonic cone. This is generally not a cone itself, but it remains a nonempty closed convex polyhedron with a unique Euclidean projection, computable through weighted isotonic regression in forecast coordinates. The gradient steps use a weighted quantile loss. The coverage criterion and calibration target are unchanged (strictly increasing invertible transformation); calibration is not guaranteed.

1.1.1 Standard MultiQT

The original MultiQT procedure [**?**] is

$$\theta_t = \Pi_{\mathcal{K}-b_t}(\tilde{\theta}_t), \qquad q_t^{\text{std}} = b_t + \theta_t, \tag{3}$$

$$\text{cov}_t^{\text{std},\alpha} = \mathbf{1}\{y_t \le q_t^{\text{std},\alpha}\}, \qquad \tilde{\theta}_{t+1}^{\alpha} = \tilde{\theta}_t^{\alpha} - \eta(\text{cov}_t^{\text{std},\alpha} - \alpha). \tag{4}$$

1.1.2 Score-threshold tracking

To make the connection explicit, first consider the scalar score-threshold update of Angelopoulos et al. [**?**]. We then recover the MultiQT update using the signed residual as the score.

The score and threshold determine the prediction set

$$\mathcal{P}_t = \{y : S_t(y) \le c_t\}, \qquad \text{err}_t = \mathbf{1}\{r_t > c_t\}. \tag{5}$$

Update according to

$$\begin{aligned} c_{t+1} &= c_t + \eta(\text{err}_t - \delta) \\ &= c_t - \eta(\mathbf{1}\{r_t \le c_t\} - \alpha). \end{aligned} \tag{6}$$

The second line follows from $\text{err}_t = 1 - \mathbf{1}\{r_t \le c_t\}$ and $\delta = 1 - \alpha$. The prediction set is obtained by score inversion. For the signed affine scores below, that set's endpoint is the outcome quantile forecast.

Holding the observed score r fixed, the subdifferential of (**??**) in the threshold is

$$\partial_z \rho_\alpha(z, r) = \begin{cases} \{-\alpha\}, & z < r, \\ [-\alpha, 1-\alpha], & z = r, \\ \{1-\alpha\}, & z > r. \end{cases} \tag{7}$$

Selecting $1 - \alpha$ at equality gives

$$g_\alpha(z; r) = \mathbf{1}\{r \le z\} - \alpha \ \in \ \partial_z \rho_\alpha(z, r). \tag{8}$$

Thus $c_{t+1} = c_t - \eta g_\alpha(c_t; r_t)$ is (**??**). The score's distance from the threshold affects the loss; the side of the threshold determines the update direction and size.

1.1.3 MultiQT in residual coordinates

For each level, define the score and its observed value by

$$S_t^{\mathrm{std},\alpha}(y) = y - b_t^\alpha, \qquad r_t^{\mathrm{std},\alpha} = y_t - b_t^\alpha. \tag{9}$$

Using θ_t^α as the score threshold gives

$$\{y : S_t^{\mathrm{std},\alpha}(y) \le \theta_t^\alpha\} = (-\infty, b_t^\alpha + \theta_t^\alpha] = (-\infty, q_t^{\mathrm{std},\alpha}]. \tag{10}$$

Consequently,

$$\mathrm{cov}_t^{\mathrm{std},\alpha} = \mathbf{1}\{y_t \le b_t^\alpha + \theta_t^\alpha\} = \mathbf{1}\{r_t^{\mathrm{std},\alpha} \le \theta_t^\alpha\}. \tag{11}$$

Substitution into (**??**) gives

$$\boxed{\tilde{\theta}_{t+1}^\alpha = \tilde{\theta}_t^\alpha - \eta(\mathbf{1}\{r_t^{\mathrm{std},\alpha} \le \theta_t^\alpha\} - \alpha).} \tag{12}$$

The original MultiQT loss has the exact residual-coordinate representation

$$\begin{aligned} \ell_t^{\mathrm{std}}(\theta) &= \sum_{\alpha\in\mathcal{A}} \rho_\alpha(b_t^\alpha + \theta^\alpha, y_t) \\ &= \sum_{\alpha\in\mathcal{A}} \rho_\alpha(\theta^\alpha, y_t - b_t^\alpha) = \sum_{\alpha\in\mathcal{A}} \rho_\alpha(\theta^\alpha, r_t^{\mathrm{std},\alpha}). \end{aligned} \tag{13}$$

The threshold c_t of the scalar score tracker corresponds to the offset θ_t^α when the score is $y - b_t^\alpha$. For a single level the constraint is all of $\mathbb{R}$, so the hidden and played states coincide and (**??**) is exactly scalar quantile tracking. For multiple levels, MultiQT uses a joint projection: evaluate g_α at the played θ_t^α, but apply the increment to the hidden $\tilde{\theta}_t^\alpha$.

1.1.4 Studentized residuals and their update

Now replace the residual score by the studentized residual.

For this construction, choose the scales from information available at issuance and hold them fixed when optimizing the threshold. Define

$$S_t^{\mathrm{stu},\alpha}(y) = \frac{y - b_t^\alpha}{\sigma_t^\alpha}, \qquad r_t^{\mathrm{stu},\alpha} = \frac{y_t - b_t^\alpha}{\sigma_t^\alpha}. \tag{14}$$

Inverting the score gives

$$\begin{aligned} S_t^{\mathrm{stu},\alpha}(y) \le u_t^\alpha \quad &\Longleftrightarrow \quad y \le b_t^\alpha + \sigma_t^\alpha u_t^\alpha, \\ q_t^{\mathrm{stu},\alpha} = b_t^\alpha + \sigma_t^\alpha u_t^\alpha, &\qquad \mathrm{cov}_t^{\mathrm{stu},\alpha} = \mathbf{1}\{r_t^{\mathrm{stu},\alpha} \le u_t^\alpha\}. \end{aligned} \tag{15}$$

Standard MultiQT tracks the outcome-unit residual threshold θ_t^α and issues $b_t^\alpha + \theta_t^\alpha$. Studentization tracks u_t^α and issues $b_t^\alpha + \sigma_t^\alpha u_t^\alpha$. The factor σ_t^α enters through the inverse score map.

Apply the same quantile-loss construction to the studentized scores:

$$\begin{aligned} \ell_t^{\mathrm{stu}}(u) &= \sum_{\alpha\in\mathcal{A}} \rho_\alpha(u^\alpha, r_t^{\mathrm{stu},\alpha}) \\ &= \sum_{\alpha\in\mathcal{A}} \frac{1}{\sigma_t^\alpha} \rho_\alpha(b_t^\alpha + \sigma_t^\alpha u^\alpha, y_t). \end{aligned} \tag{16}$$

Using (**??**), its selected subgradient is

$$g_t^{\mathrm{stu},\alpha}(u_t) = \mathbf{1}\{r_t^{\mathrm{stu},\alpha} \le u_t^\alpha\} - \alpha = \mathrm{cov}_t^{\mathrm{stu},\alpha} - \alpha. \tag{17}$$

The chain rule gives the same result. Away from ties, for one level,

$$\frac{\partial}{\partial u^\alpha}\left[\frac{\rho_\alpha(b_t^\alpha+\sigma_t^\alpha u^\alpha, y_t)}{\sigma_t^\alpha}\right] = \frac{1}{\sigma_t^\alpha}(\mathbf{1}\{y_t \le b_t^\alpha + \sigma_t^\alpha u^\alpha\} - \alpha)\sigma_t^\alpha$$
$$= \mathbf{1}\{r_t^{\mathrm{stu},\alpha} \le u^\alpha\} - \alpha. \tag{18}$$

At a tie use the selected subgradient in (**??**). The inverse scale in the loss cancels the scale in the inverse score map.

Retaining MultiQT's lazy update therefore gives

$$\boxed{\tilde{u}_{t+1}^\alpha = \tilde{u}_t^\alpha - \eta g_t^{\mathrm{stu},\alpha}(u_t) = \tilde{u}_t^\alpha - \eta\left(\mathbf{1}\left\{\frac{y_t - b_t^\alpha}{\sigma_t^\alpha} \le u_t^\alpha\right\} - \alpha\right).} \tag{19}$$

Initialize $\tilde{u}_1 = 0$. The score is compared with the *played* threshold; the increment is applied to the *hidden* threshold. The projection that produces u_t is specified next.

	Standard MultiQT	Studentized extension
Observed score	$r_t^{\mathrm{std},\alpha} = y_t - b_t^\alpha$	$r_t^{\mathrm{stu},\alpha} = (y_t - b_t^\alpha)/\sigma_t^\alpha$
Gradient	$\mathbf{1}\{r_t^{\mathrm{std},\alpha} \le \theta_t^\alpha\} - \alpha$	$\mathbf{1}\{r_t^{\mathrm{stu},\alpha} \le u_t^\alpha\} - \alpha$
Update origin	$\tilde{\theta}_t^\alpha$	$\tilde{u}_t^\alpha$
Issued forecast	$b_t^\alpha + \theta_t^\alpha$	$b_t^\alpha + \sigma_t^\alpha u_t^\alpha$

The gradient has the same form because both losses apply quantile loss to a score threshold. The observed scores, state units, and issued forecasts change. An unnormalized outcome-unit loss would instead have gradient $\sigma_t^\alpha(\mathrm{cov}_t^{\mathrm{stu},\alpha} - \alpha)$ with respect to u^α, and would define a different update.

1.1.5 Ordering constraints and projection

The constraint always concerns the outcome forecasts: $q_t^{\alpha_1} \le \cdots \le q_t^{\alpha_m}$. For the ordinary residual score, this requires

$$b_t + \theta \in \mathcal{K} \iff \theta \in C_t^{\mathrm{std}} := \mathcal{K} - b_t, \tag{20}$$
$$\theta^{\alpha_i} - \theta^{\alpha_{i+1}} \le b_t^{\alpha_{i+1}} - b_t^{\alpha_i}, \qquad i = 1, \ldots, m-1. \tag{21}$$

For studentization, $q_t^{\mathrm{stu}} = b_t + A_t u_t$, so

$$\boxed{C_t^{\mathrm{stu}} = \{u \in \mathbb{R}^m : b_t + A_t u \in \mathcal{K}\} = A_t^{-1}(\mathcal{K} - b_t).} \tag{22}$$

This is a closed convex polyhedron given by linear inequalities. It contains zero when $b_t \in \mathcal{K}$. The shift means that, like $\mathcal{K} - b_t$, it is generally not a cone. For unequal scales, $A_t^{-1}\mathcal{K} \neq \mathcal{K}$ in general, so the set cannot be simplified to $\mathcal{K} - A_t^{-1}b_t$. The standard constraint $b_t + \theta \in \mathcal{K}$ is replaced by $b_t + A_t u \in \mathcal{K}$, and the Euclidean projection is well defined:

$$u_t = \Pi_{C_t^{\mathrm{stu}}}(\tilde{u}_t), \qquad q_t^{\mathrm{stu}} = b_t + A_t u_t. \tag{23}$$

Substituting $u = A_t^{-1}(q - b_t)$ into the projection objective yields

$$q_t^{\mathrm{stu}} = \arg\min_{q \in \mathcal{K}} \|A_t^{-1}(q - b_t) - \tilde{u}_t\|^2$$
$$= \arg\min_{q \in \mathcal{K}} \sum_{\alpha \in \mathcal{A}} \frac{(q^\alpha - b_t^\alpha - \sigma_t^\alpha \tilde{u}_t^\alpha)^2}{(\sigma_t^\alpha)^2}. \tag{24}$$

In weighted-projection notation, the exact identity is

$$\boxed{b_t + A_t \Pi_{A_t^{-1}(\mathcal{K}-b_t)}(\tilde{u}_t) = \Pi_{\mathcal{K}}^{A_t^{-2}}(b_t + A_t \tilde{u}_t).} \tag{25}$$

Thus the implementation is weighted isotonic regression of $v_t^\alpha = b_t^\alpha + \sigma_t^\alpha \tilde{u}_t^\alpha$ with weights $1/(\sigma_t^\alpha)^2$.

The ordinary residual case has forecast-space metric I; the studentized case has metric A_t^{-2}. These weights follow from the chosen Euclidean geometry of the tracked score coordinates.

1.1.6 Common and constant scales

For $S_t^\alpha(y) = (y - b_t^\alpha)/\sigma_t$, we have $A_t = \sigma_t I$. Positive scalar multiplication preserves $\mathcal{K}$:

$$C_t^{\text{stu}} = \mathcal{K} - b_t/\sigma_t, \qquad q_t^{\text{stu}} = \Pi_{\mathcal{K}}(b_t + \sigma_t \tilde{u}_t). \tag{26}$$

All projection weights are equal and cancel. Equivalently, run standard MultiQT on base b_t/σ_t and outcome y_t/σ_t, then multiply the issued forecast by σ_t.

If $\sigma_t = \sigma$ is constant, standard MultiQT on the original inputs is also equivalent after converting the state and learning rate:

$$\tilde{\theta}_t = \sigma \tilde{u}_t, \qquad \eta_{\text{std}} = \sigma \eta_{\text{stu}}. \tag{27}$$

1.2 Questions and ideas

- What other scores would be reasonable to consider, i.e. what property of the base forecast do I need to recalibrate? Instead of asking for 1d PIT calibration, could I ask for 1d hdr-calibration? With studentized scores, or in general strictly increasing and invertible scores, the coverage indicator and calibration target do not change

2 Coding Updates

2.1 9 November 2026

2.1.1 Feedback handling: Stateful, re-run and delay

The feedback design is independent of the recalibration method and uses the report and cutoff conventions in Notation.

Stateful (learn once). Wait d calendar weeks from the first valid report. At the next forecast cutoff ι_t with $t > s$, look up the latest report. If it is invalid, wait for a later forecast; never substitute an older report.

$$m_s^{(d)} = \nu_s + 7d \text{ days}, \tag{28}$$

$$\tau_d(s) = \min\{t > s : \iota_t \geq m_s^{(d)},\ y_s^{[\iota_t]} \text{ is valid}\}, \qquad \mathcal{R}_t^{(d)} = \{s : \tau_d(s) = t\}. \tag{29}$$

An empty $\mathcal{R}_t^{(d)}$ means no update. For each released row, use $y_s^{[\iota_t]}$. If a missing forecast postpones the update so that $\iota_t > m_s^{(d)}$, select the report at ι_t, rather than fixing it at $m_s^{(d)}$. Feedback for each q_s is used at most once; revisions after that update are ignored.

For plain MultiQT on counts, start at $\tilde{\theta}_0 = 0$; when learning is active, update using previously issued q_s^α, then issue q_t:

$$\tilde{\theta}_t^\alpha = \tilde{\theta}_{t-1}^\alpha - \eta_t \sum_{s \in \mathcal{R}_t^{(d)}} (\mathbf{1}\{y_s^{[\iota_t]} \leq q_s^\alpha\} - \alpha), \tag{30}$$

$$\theta_t = \Pi_{\mathcal{K}-b_t}(\tilde{\theta}_t), \qquad q_t = b_t + \theta_t. \tag{31}$$

Sum with η_t; keep the hidden state unprojected. For log tracking, apply the same update in ψ coordinates. The default retrospective initialization of S_0 is described in Section **??**.

Weekly re-run. At each reference-week index t, re-run MultiQT from the start using the latest reports $y_s^{[\iota_t]}$. Only the new forecast $q_t^{[t]}$ is issued; previously issued forecasts remain unchanged. Starting from $\tilde{\theta}_0^{[t]} = 0$, replay $j = 1, \ldots, t$ with feedback only at $j = \tau_d(s)$, retaining delay d; omit invalid reports:

$$\tilde{\theta}_j^{\alpha,[t]} = \tilde{\theta}_{j-1}^{\alpha,[t]} - \eta_j^{[t]} \sum_{\substack{s \in \mathcal{R}_j^{(d)} \\ y_s^{[\iota_t]} \text{ valid}}} (\mathbf{1}\{y_s^{[\iota_t]} \leq q_s^{\alpha,[t]}\} - \alpha), \tag{32}$$

$$\theta_j^{[t]} = \Pi_{\mathcal{K}-b_j}(\tilde{\theta}_j^{[t]}), \qquad q_j^{[t]} = b_j + \theta_j^{[t]}. \tag{33}$$

For each case and feedback delay, estimate S_0 and the reference expert loss L_0 once from the first 20 feedback rows consumed by the stateful run. Every weekly re-run reuses these constants. With warmup replay enabled, the constants are applied before the 20th feedback row is released, making initialization retrospective. See Section **??** for S_0 and initialization timing, and Section **??** for L_0.

Studentized methods use $\tilde{u}$, q^{stu} and projection (**??**), issuing the base until their scale is available; re-runs reconstruct the scale history. Evaluation uses a common latest-report truth, separate from learning.

2.1.2 Tuning and initialization

The reference scales are estimated from $I_{20}^{(d)}$ and its stateful feedback values $\bar{y}_s^{(d)}$; these 20 rows need not span 20 calendar weeks. Method parameters, such as a rate multiplier or an

expert-weighting lookback, are selected on a tuning period. These are distinct steps in the implementation. The expert reference loss L_0 and its effect on weight updates are described in Section **??**.

Initial error scale S_0. Pool absolute base errors across the reference rows and all quantile levels, and take their empirical 90th percentile:

$$S_0 = \max\Big\{\varepsilon_S, Q_{0.9}\Big(\Big\{|\psi(b_s^\alpha) - \psi(\bar{y}_s^{(d)})| : s \in I_{20}^{(d)},\ \alpha \in \mathcal{A}\Big\}\Big)\Big\}. \tag{34}$$

Here $\varepsilon_S = 10^{-6}$ is a numerical floor. The fixed-rate tracker uses $\eta = \kappa S_0$, measured in tracker units. The studentized rate is instead chosen directly in standardized units. In the code, S_0 also sets the studentized-scale bounds $[0.05S_0, S_0]$.

Retrospective initialization. The default `replay_warmup = TRUE` computes both constants from the eventual first 20 releases, then applies them from the start. Forecasts before the 20th release can therefore depend on information unavailable at their issue time through these constants. This initialization is retrospective even though feedback updates follow the schedule above. With replay disabled, learning waits until the warmup is complete; if fewer than 20 rows are released, learning remains inactive. Once estimated, S_0 and L_0 remain fixed across weekly re-runs. Corrections, expert weights and time-varying studentized scales are still reconstructed in each re-run. The reference scales are computed by `error_scale()` and `weighting_loss_scale()` in `R/03_functions.R`.

Tuning procedure. `R/05_tuning.R` runs COVID–`log1p`, COVID–`identity`, flu–`log1p`, then flu–`identity`. For each pair, it completes stateful delay 0, stateful delay 1, then weekly re-run with delay 0. Parameters are chosen independently for these 12 configurations, with eight stages each (96 stages in the full schedule).

The `log1p` tracker updates on $\log(1 + \text{counts})$ and minimizes quantile loss on that scale. The `identity` tracker updates on raw counts and minimizes quantile loss per 100,000 residents. For each candidate setting, forecasts run online under the chosen feedback design; only tuning weeks are scored, against the latest reports. Mean losses are pooled equally over location–horizon cases to select one value per method and configuration. Choose the smallest loss, with the smaller parameter value at ties. The criterion is not divided by L_0; calibration error is reported separately.

Stage	What is tuned, and what happens next
1	Tune the ordinary trackers and expert mixtures: tracker rates, weighting lookback H, fixed restart-weighting rate, and diverse experts' slow rate. Search one setting at a time for each method, then retain its best value.
2–7	Tune the soft switch in order: strength, threshold, width, then the forecast-path, nowcast and revision signal weights. Keep each selected value when moving to the next setting, using the expert bank tuned in stage 1.
8	Tune the outer MultiQT correction rate on each already tuned inner mixture. Then reset to defaults for the next disease–tracker–feedback configuration.

All prepared locations contribute. The hub design tunes on horizons $-1, \ldots, 3$ through 31 May 2025; the seasons design uses horizons $0, \ldots, 3$, with -1 reusing those choices. Configurations without usable tuning data skip the remaining stages.

Candidate grids. The following grids are used in `R/05_tuning.R` for each disease–tracker–feedback configuration. Setting names match the code.

Stage	Setting	Candidate values
1	`fixed_multiplier`	0.001, 0.0025, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2, 0.4
1	`slope_min_multiplier`	0.002, 0.004, 0.01, 0.02, 0.05, 0.1
1	`median_rate`	0.003, 0.01, 0.03, 0.1, 0.3, 1
1	`H`	2, 4, 8, 16, 32
1	`restart_lambda`	0.25, 0.5, 1, 2.72, 7, 20
1	`diverse_slow_multiplier`	0.001, 0.002, 0.004, 0.008, 0.02, 0.05, 0.1, 0.2
2	`switch_strength`	0, 0.25, 0.5, 0.75, 1
3	`switch_threshold`	0.02, 0.05, 0.1, 0.2
4	`switch_width`	0.02, 0.05, 0.1, 0.2
5	`switch_weight_path`	0, 0.5, 1
6	`switch_weight_nowcast`	0, 0.5, 1
7	`switch_weight_revision`	0, 0.5, 1
8	`outer_multiplier`	0.001, 0.002, 0.004, 0.01, 0.02, 0.05, 0.1, 0.2

The four `multiplier` grids scale S_0; the slope multiplier controls only the floor term of its capped, slope-dependent rate. `median_rate` is a step in standardized units, H counts weight updates, and `restart_lambda` is a dimensionless rate for losses normalized by L_0. The switch settings control the gate.

Single-pass sequential grid search. At each search, vary one setting and select its lowest-loss candidate, holding all other settings fixed. Retain choices from earlier stages; settings not yet tuned keep their defaults. Stage 1 searches the different methods separately; stages 2–7 successively tune the six settings of the same soft-switch method; stage 8 tunes each outer correction on its already tuned inner mixture.

For the six soft-switch settings, a full Cartesian search would evaluate

$$5 \times 4 \times 4 \times 3 \times 3 \times 3 = 2160$$

joint combinations. The sequential search instead makes

$$5 + 4 + 4 + 3 + 3 + 3 = 22$$

candidate evaluations per location–horizon case, with some configurations possibly evaluated again. These counts cover only stages 2–7; all 12 disease–tracker–feedback configurations are still tuned separately. This is one pass: earlier choices are not revisited after later settings change. It is cheaper, but parameter interactions can make the result depend on the search order, and the best joint combination is not guaranteed.

2.1.3 Effect of increasing feedback delay

2.1.4 US and the five largest states

2.1.5 Extended data history

2.1.6 Trade-off and evaluation plots, including forecast curves

2.1.7 Studentized scores and non-additive updates

2.2 Weighting and adaptive weighting of experts

An expert is a complete forecasting procedure, such as MultiQT with a particular rate or a restarted tracker. The mixture learns which experts to trust from their losses on released outcomes. Even a fixed learning rate gives changing weights; adaptive weighting additionally changes the learning rate. Each location–horizon case is learned separately. The formulas below first consider a fixed bank whose experts are all eligible for every update.

2.2.1 Mixtures and feedback losses

With weights specific to each quantile level, issue

$$\hat{q}^{\alpha} = \sum_{k=1}^{K} w_k^{\alpha} f_k^{\alpha}, \qquad w_k^{\alpha} \geq 0, \quad \sum_{k=1}^{K} w_k^{\alpha} = 1, \qquad q = \Pi_{\mathcal{K}}(\hat{q}). \tag{35}$$

Different levels can favor different experts. This can introduce crossings, so the code repairs the mixture by isotonic projection. With common weights, replace w_k^{α} by w_k at every level. If every expert's quantiles are ordered, this common-weight mixture is already ordered and needs no repair.

On receiving feedback, score the expert forecasts for the released row:

$$L_{k,n}^{\alpha} = \rho_{\alpha}\Big(f_{k,s(n)}^{\alpha}/c_{\text{scale}}, Y_n/c_{\text{scale}}\Big), \qquad L_{k,n} = \frac{1}{m}\sum_{\alpha\in\mathcal{A}} L_{k,n}^{\alpha}, \tag{36}$$

$$\ell_{k,n}^{\alpha} = L_{k,n}^{\alpha}/L_0, \qquad \ell_{k,n} = \frac{1}{m}\sum_{\alpha\in\mathcal{A}} \ell_{k,n}^{\alpha}. \tag{37}$$

The reference L_0 is estimated as in Section **??** and is common to all experts and levels within the case and feedback design. In stateful learning, score forecasts actually issued for that row; a re-run instead scores the forecasts reconstructed inside that re-run. Feedback timing follows Section **??**.

2.2.2 Exponential updates and optional sharing

Before optional sharing, update separately at each level:

$$\bar{w}_{k,n+1}^{\alpha} = \frac{w_{k,n}^{\alpha}\exp(-\lambda_n^{\alpha}\ell_{k,n}^{\alpha})}{\sum_{j=1}^{K} w_{j,n}^{\alpha}\exp(-\lambda_n^{\alpha}\ell_{j,n}^{\alpha})}. \tag{38}$$

With common weights, there is one update using the mean loss:

$$\bar{w}_{k,n+1} = \frac{w_{k,n}\exp(-\lambda_n\ell_{k,n})}{\sum_{j=1}^{K} w_{j,n}\exp(-\lambda_n\ell_{j,n})}. \tag{39}$$

An expert with lower loss gains weight relative to one with higher loss. A larger learning rate gives a stronger response to the same loss gap. The initial weights are uniform. The option $\lambda = 0$ keeps a uniform average of the active experts.

When sharing is enabled, follow the exponential step by

$$w_{k,n+1}^{\alpha} = (1-\gamma)\bar{w}_{k,n+1}^{\alpha} + \gamma/K. \tag{40}$$

The common-weight version again omits α. This gives previously downweighted experts a chance to recover when conditions change. The default is $\gamma = 1/(2H)$; without sharing, $\gamma = 0$. This sharing among experts is separate from choosing whether a weight vector is shared across quantile levels.

2.2.3 Adaptive learning rates

The first H processed updates use λ_{start}. Record each update's second moment using its weights before the exponential step and sharing:

$$M_n^\alpha = \sum_{k=1}^{K} w_{k,n}^\alpha (\ell_{k,n}^\alpha)^2, \qquad M_n = \sum_{k=1}^{K} w_{k,n} \ell_{k,n}^2. \tag{41}$$

For per-level weighting, the rate is

$$\lambda_n^\alpha = \begin{cases} \lambda_{\text{start}}, & 1 \le n \le H, \\ \sqrt{\dfrac{\log(2KH)+1}{\sum_{r=n-H}^{n-1} M_r^\alpha}}, & n > H. \end{cases} \tag{42}$$

Thus update $H+1$ is the first to use an adaptive rate. Only the previous H moments enter, excluding the loss being processed now. If their sum is zero, retain the previous finite rate. Larger recent second moments give a smaller rate, moderating the response to large losses.

With common weights, use (**??**) without α and with M_r. Each expert's loss is averaged over levels *before* it is squared in (**??**); the result is not the mean of the per-level second moments. Per-level weighting maintains a separate weight vector, moment history and adaptive rate for each α. Common weighting maintains one of each and learns a compromise across levels; its weights are not an average of separately learned per-level weights.

Later starts and implementation. For restart banks, only experts that issued the released forecast are eligible for its update. Apply the formulas to their weights normalized to sum to one, then restore their original total mass; other experts keep their mass. Use the number of eligible experts in the rate formula. Each newly admitted expert receives $1/K$ of the enlarged active bank's mass, preserving the relative weights of incumbents. Updates with fewer than two eligible experts do not count toward H. These rules are implemented in `exp_weights()` in `R/03_functions.R`. The formulas describe ordinary weighting. The phase-specific extension can also multiply losses by an issuance-time context weight; its second moments use those multiplied losses, zero-weight feedback is skipped, and its sharing fraction is adjusted to the same exposure.

2.2.4 Expert loss normalization

Using the quantile loss in (**??**), define one reference loss per case and delay:

$$L_0 = \max\left\{ \varepsilon_L, \frac{1}{20|\mathcal{A}|} \sum_{s \in I_{20}^{(d)}} \sum_{\alpha \in \mathcal{A}} \rho_\alpha\left(\frac{b_s^\alpha}{c_{\text{scale}}}, \frac{\bar{y}_s^{(d)}}{c_{\text{scale}}} \right) \right\}, \qquad \varepsilon_L = 10^{-6}. \tag{43}$$

Previously, expert weighting used unnormalized quantile losses. Now (**??**) expresses every expert's loss relative to this initial base loss. A value of 0.8 means 80% of the base's initial mean loss, not necessarily a 20% improvement over the base in the current week. Normalized losses can exceed one. These weighting losses remain on the natural count scale even for a log tracker.

Initial and fixed-rate weight updates. The exponent in an update is $-\lambda L_{k,n}^\alpha / L_0$ for per-level weights, or $-\lambda L_{k,n} / L_0$ for common weights. The common divisor preserves expert rankings but makes the response depend on loss gaps relative to initial base performance. Quantile loss scales linearly with a positive change of units; the current loss and L_0 therefore scale together. The normalized losses and weight path are unchanged, provided the numerical floor is inactive.

This matters throughout a fixed-rate run and during the first H updates of a learned-rate run, which use λ_{start}. Those H updates are distinct from the 20 feedback rows used to estimate

L_0. For example, two equally weighted experts with losses 80 and 100, $L_0 = 100$ and $\lambda_{\text{start}} = 1$ receive weights about 0.55 and 0.45 before sharing. Using those raw losses with the same numerical starting rate would put almost all weight on the first expert after one update.

Effect on adaptive learning rates. After H updates, the rule in (**??**) already adapts to loss magnitude. Conditional on identical previous weights and a positive moment sum, normalization gives, separately at each level,

$$\begin{aligned} M_r^{\alpha,\text{norm}} &= M_r^{\alpha,\text{raw}}/L_0^2, & \lambda_n^{\alpha,\text{norm}} &= L_0\lambda_n^{\alpha,\text{raw}}, \\ \lambda_n^{\alpha,\text{norm}}\ell_{k,n}^{\alpha} &= \lambda_n^{\alpha,\text{raw}}L_{k,n}^{\alpha}. \end{aligned} \tag{44}$$

The same identity holds for common weights after omitting α. The reciprocal changes in the adaptive rate and loss cancel, so the exponential update is the same under that comparison. If $L_0 < 1$, normalization increases losses and decreases the adaptive rate; if $L_0 > 1$, it does the reverse. This does not make two runs with the same numerical starting rate identical: normalization can change their first H weight updates, and those weights affect both later second moments and subsequent updates. Equivalent raw and normalized runs require $\lambda_{\text{start}}^{\text{raw}} = \lambda_{\text{start}}^{\text{norm}}/L_0$ and otherwise matching settings. If a moment sum is zero, the code retains the previous rate; equivalence then also requires those retained rates to be scaled alike.

We keep L_0 normalization throughout learned-rate runs. It standardizes initialization and keeps stored moments and incoming losses in the same units. Switching it off after H updates without converting the stored moments and rates would make those units inconsistent. For learned rates, it is not an additional reduction in update strength once the adaptive rule compensates for it.

Effect across quantile levels and re-runs. The single L_0 is pooled across levels; it is not a separate baseline for each α and does not equalize their typical losses. With per-level weights, $\ell_{k,n}^{\alpha}$ drives that level's own update, and its own second moments determine λ_n^{α} after initialization. With common weights, the mean $\ell_{k,n}$ drives one update and one adaptive rate, as described in Section **??**. During a re-run, losses use the current reports and reconstructed expert forecasts, while L_0 stays at its stateful value. Normalization sets the scale of learning; it does not guarantee better accuracy or calibration, and evaluation losses are not divided by L_0.

2.2.5 Expert combinations implemented in the code

Table **??** collects the 12 mixtures and five outer-MultiQT variants selected by `apply_methods()` in `R/03_functions.R` for scripts 04 and 05. Adaptive rates follow (**??**); fixed rates are tuned before evaluation and then held constant within a run. Both update the expert weights. Equal averaging uses $\lambda = 0$, so weight learning and sharing are inactive. "Per quantile" means a separate weight vector at each level; "common" means the same learned vector at every level.

Table 1: Current expert combinations. Bank names and outer corrections are described below. "Sharing" redistributes mass among experts.

Method ID	Expert bank	Weighting rate	Sharing	Level weights
Persistent rate bank				
`uniform`	Rate	Equal	–	Identical
`weighted`	Rate	Adaptive	No	Per quantile
`weighted_common`	Rate	Adaptive	No	Common
`shared`	Rate	Adaptive	Yes	Per quantile
`shared_common`	Rate	Adaptive	Yes	Common
Weekly restart banks				
`restart_uniform`	Restart–1	Equal	–	Identical
`restart_weighted`	Restart–1	Fixed, tuned	Yes	Per quantile
`restart_grid`	Restart grid	Fixed, tuned	Yes	Per quantile
Diverse bank and context-dependent combination				
`diverse_uniform`	Diverse	Equal	–	Identical
`diverse_shared`	Diverse	Adaptive	Yes	Per quantile
`diverse_shared_common`	Diverse	Adaptive	Yes	Common
`diverse_soft_switch`	Diverse	Adaptive in each context	Yes	Common in each context
Mixtures followed by an outer MultiQT				
`shared_outer`	Rate	Adaptive	Yes	Per quantile
`shared_common_outer`	Rate	Adaptive	Yes	Common
`restart_uniform_outer`	Restart–1	Equal	–	Identical
`restart_weighted_outer`	Restart–1	Fixed, tuned	Yes	Per quantile
`restart_grid_outer`	Restart grid	Fixed, tuned	Yes	Per quantile

Expert banks. The following bank sizes and numerical values are the current defaults in `multiqt_settings()` and `method_parameters()`. Tuned values can differ by method, disease, tracker and feedback design.

- **Rate:** 11 persistent MultiQT experts, all starting in the first round, with $\eta = \kappa S_0$ and $\kappa \in \{0, .01, .015, .02, .03, .05, .075, .1, .15, .2, .3\}$. The zero-rate expert reproduces the base forecast.
- **Restart–1 and restart grid:** fresh MultiQT copies start at every issue week after warmup; with successful initialization and warmup replay, starts begin in the first round. Restart–1 uses $\kappa = .05$; the grid uses $\kappa \in \{.01, .05, .2\}$ at every start. Each copy starts from the base and learns only from feedback on its own forecasts. These banks grow over time and have no permanent zero-rate base expert.

- **Diverse:** eight experts: the base; slow MultiQT; slow MultiQT with a leaky shift; three growth-adjusted starting forecasts with damping coefficients $-.5, .5, 1$; and two width-adjusted forecasts with factors $.75, 1.5$ around the base median. Every non-base expert has its own slow MultiQT correction, with default $\eta = .008S_0$. Growth adjustments use $\log(1 + x)$ and the latest released report; the negative damping coefficient extends the implied growth. Width adjustments use tracker units. The three ordinary diverse mixtures can have different tuned slow rates.

How phase information enters. `diverse_soft_switch` learns two common-weight distributions over the same diverse experts. At issuance, it combines them as

$$w_{k,t}^{\text{mix}} = (1 - G_t)w_{k,t}^{B} + G_t w_{k,t}^{D}. \tag{45}$$

The gate uses information available at that issue cutoff: the shape of the base forecast path, changes in nowcasts, and changes between forecasts for the same target. It describes declining context rather than a known peak date or a probability of having passed the peak. When feedback for row $s(n)$ arrives, the baseline and declining banks use losses multiplied by $2(1 - G_{s(n)})$ and $2G_{s(n)}$, respectively. The saved issuance-time gate determines this update; each bank has its own adaptive rate and context-scaled moments. With switch strength zero, $G_t = .5$ and the method equals `diverse_shared_common` when their settings match. By default, the soft switch inherits that comparator's tuned expert bank.

Outer corrections and the restart control. Each `_outer` method takes the mixture with the same name without that suffix and adds a further MultiQT correction. The table describes its inner mixture. The outer tracker learns from its own final forecasts, with continuous offsets even when new restart experts join. Its default rate is $.004S_0$, tuned after the inner mixture. The related `restart_persistent` control keeps only the first Restart–1 expert; it is a single tracker, not an expert combination.

Which settings are tuned. The four learned rate-bank mixtures tune H. The two learned restart mixtures tune their fixed `restart_lambda`; the default is 2.72. The ordinary diverse mixtures tune their slow tracker rate. Soft-switch tuning selects gate strength, threshold, width and three signal weights in sequence. Outer methods tune their added correction rate. Adaptive weighting starts at $\lambda_{\text{start}} = 2.72$ with default $H = 8$ unless overridden; sharing uses $\gamma = 1/(2H)$. Equal weights remain identical across levels even though the code stores them in per-level columns. All combinations use the same loss and feedback conventions described above.

References

[1] Tiffany Ding, Isaac Gibbs, and Ryan J. Tibshirani. *Calibrated Multi-Level Quantile Forecasting.* arXiv:2512.23671v1, 2025. MultiQT paper.

[2] Anastasios N. Angelopoulos, Emmanuel J. Candès, and Ryan J. Tibshirani. *Conformal PID Control for Time Series Prediction.* Advances in Neural Information Processing Systems 36, 2023. NeurIPS paper.

[3] CMU Delphi. *Delphi hub forecasting: calibration experiments and tracker code.* Pinned source revision `9abebcfb`, inspected 9 November 2026. E09: constant learning rate and revision delay; tracker implementation.